Project Description

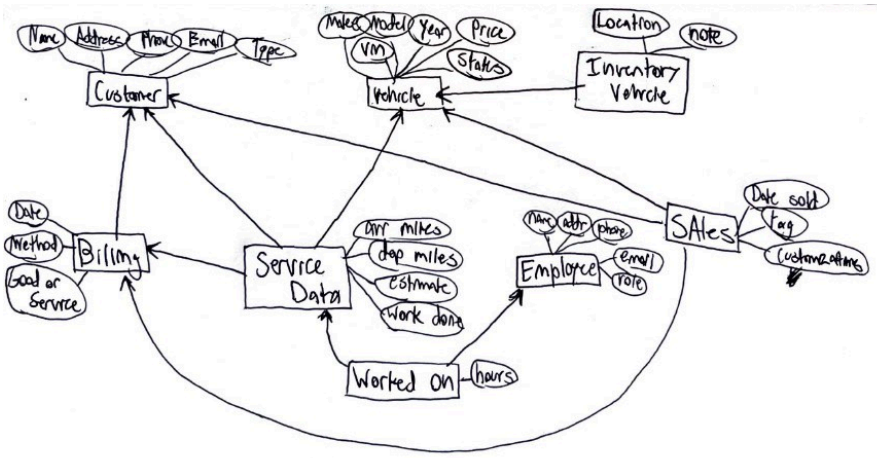
The goal of this project is to simulate the full database development lifecycle. This means implementing every part of a database that's used in a real-life scenario; in this case, a car dealership. This includes:

- Implementing a database schema that is relevant to the scenario
- Creating accounts for employees so they can access the database
- Creating roles to assign to employees within the DBMS so they can edit only what they need
    - Mechanics can update only service data
    - Salespeople can view only vehicle inventory and sales
    - Billing staff can manage only payment information
- Ensure that the database system works intuitively (for non-technical users) and effectively (ensure data integrity and minimize redundancy in data and user interaction with the DBMS)

It should be noted that the scope of this project is not all-encompassing. For example, the web server is only running on localhost, not fully deployed online; and the project uses SQLite, which is not the ideal database system for larger-scale scenarios. However, our implementation attempts to address just about every part of the database development cycle that is relevant to this course.

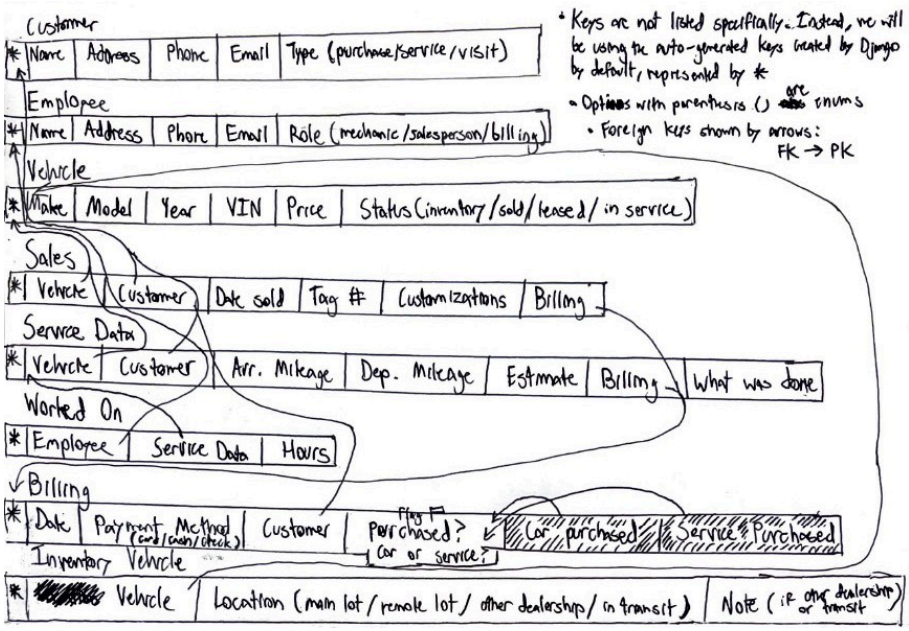
Technical Details

ER Diagram



Our ER Diagram is taken largely from the rubric, with "worked on" being the many-to-one relationship of employees to a service, with each employee's hours worked on that service also being tracked. Employee data like name and email is stored in a django user account (mentioned later).

Relational Schema



Notes read as follows:

- Keys are not listed specifically. Instead, we will be using the auto-generated keys created by Django by default, represented by *
- Options with parenthesis are enums
- Foreign Keys are shown by arrows (FK -> PK)

Software and Frameworks

Our implementation uses Django, a Python-based web development framework that is focused on making development fast and simple. It takes care of the entire back-to-front end of development, and, crucially, comes with a secure admin panel built-in, so we didn't have to worry about developing a web interface and user authentication completely on our own. It also ties in with SQLite by default, meaning that we do not need to configure SQL queries on our own in python or a separate middleman language.

Important Scripts

The most important scripts created are models.py and admin.py.

Models.py

Django abstracts away SQL queries and handles them internally. Instead, to handle a relation, you must create what Django calls a "model", which simply defines the relation in terms of Django-handle-able data types. These are all stored in the models.py file, making it quite important to the project. Here's the customer relation implemented in a Python Django model:

```python
models.py 2
django-project > models > models.py > ...
from django.db import models

class Customer(models.Model):
    name = models.CharField(max_length=60)
    address = models.CharField(max_length=255)
    email = models.EmailField(max_length=60)
    phoneNumber = models.CharField(max_length=10)

    class CustomerType(models.TextChoices):
        PURCHASE = 'Purchase',
        SERVICE = 'Service',
        VISIT = 'Visit',

    customerType = models.CharField(
        max_length=8,
        choices=CustomerType.choices,
        default=CustomerType.SERVICE,
    )
```

For reference, here's its equivalent MySQL CREATE TABLE statement:

```sql
CREATE TABLE customer (
    id          INT             NOT NULL AUTO_INCREMENT,
    name        VARCHAR(60)     NOT NULL,
    address     VARCHAR(255)    NOT NULL,
    email       VARCHAR(60)     NOT NULL,
    phoneNumber VARCHAR(10)     NOT NULL,
    customerType ENUM('Purchase', 'Service', 'Visit') NOT NULL DEFAULT 'Service',
    PRIMARY KEY (id)
);
```

Another thing to note is that Django automatically takes care of making a primary key for every tuple, and that the last attribute in this example, customerType, is an enum. Both of these are reflected in the MySQL statement, where the primary key is set as an auto-incrementing attribute, and customerType is an enum.

Admin.py

Admin.py is also quite important, because that's where all of the models are configured to show in the admin panel (called registering). Without it, the only way to add and edit entries in the database would be through the shell, which is not ideal practically.

Here's that same customer model being registered in the admin panel (using the import/export and djangoql search plugins):

```
@admin.register(Customer)
class CustomerAdmin(ImportExportModelAdmin, DjangoQLSearchMixin,
admin.ModelAdmin):
    list_display = get_fields(Customer)
```

Page Rendering

Django does the backend and the frontend together. Instead of using two different tools for the backend and dynamic frontend, like Flask and PHP for example, Django does it all. In this example, the url pattern is set up in urls.py so that localhost/, with ‘’ meaning no additional text in the path, will use the view views.index. Views.index is set up to render index.html, but with some additional data about the customers. For this example, it passes all of the customer objects, which may be appropriate for a certain admin user. The html uses jinja2 tags to create a row in the table for every customer object. Django renders this page and parses the jinja2 tags on the server side before sending the page to the client.

Urls.py

```
from . import views

urlpatterns = [
    path('', views.index),
    path('admin/', admin.site.urls),
]
```

Views.py

```
from models.models import Customer

def index(request):
    context = {
        'customers': Customer.objects.all(),
    }
    return render(request, 'index.html', context)
```

Index.html

```html
<table>
    <thead>
        <tr>
            <th>Name</th>
            <th>Address</th>
            <th>Email</th>
            <th>Phone Number</th>
            <th>Customer Type</th>
        </tr>
    </thead>
    <tbody>
        {% for c in customers %}
        <tr>
            <td>{{ c.name }}</td>
            <td>{{ c.address }}</td>
            <td>{{ c.email }}</td>
            <td>{{ c.phoneNumber }}</td>
            <td>{{ c.customerType }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
```

Rendered website

Name	Address	Email	Phone Number	Customer Type
Foo	Bar	foo@bar.com	5005005000	Purchase
Bar	Foo	foo.bar@gmail.com	4004004000	Purchase
Test3	addressss	test@test.net	2002002000	Service

Security

SQL injection is not a concern with the Django admin:
“Django’s querysets are protected from SQL injection since their queries are constructed using query parameterization. A query’s SQL code is defined separately from the query’s parameters. Since parameters may be user-provided and therefore unsafe, they are escaped by the underlying database driver”.

In terms of security, Django handles passwords in a secure and complete manner:
“By default, Django uses the PBKDF2 algorithm with a SHA256 hash, a password stretching mechanism recommended by NIST. The password attribute of a User object is a string in this format: <algorithm>$<iterations>$<salt>$<hash>”.

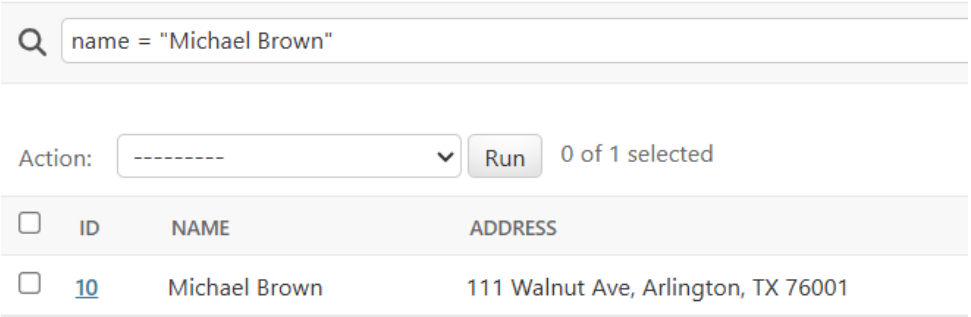
In terms of authorization, Django also handles that for us. Admin users can be assigned different levels of authority, through what Django calls “permissions” and “groups”. Permissions are “*binary (yes/no) flags designating whether a user may perform a certain task*”, such as editing existing records in a table, or adding new ones, or being able to see the table at all. Groups are “*a generic way of applying labels and permissions to more than one user*”. They are particularly helpful in this project, because they allow us to implement the permission for different types of employees (mechanics, salespeople, and billing staff) easily.

Implemented Functions

Admin Page

The djangoql plugin is included in our project because it allows for querying a relation in a user-friendly way within the admin panel. This will be shown during our demonstration along with the SQL queries.

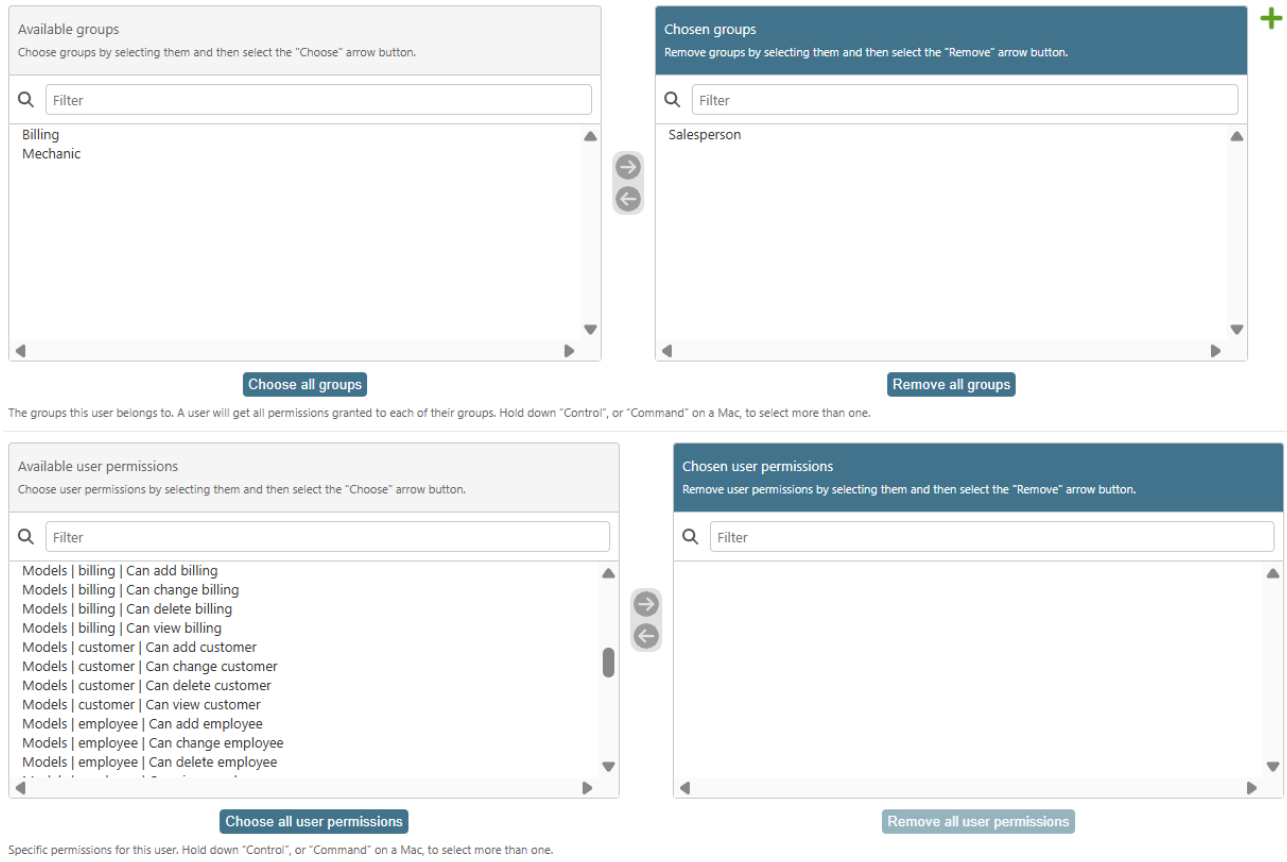
Example DjangoQL prompt:



In terms of employees, the User relation in django is not enough to store all of our relevant employee information, so we instead use a separate relation that is a specialization of User. It uses a OneToOneField (a special django FK relationship that ensures a User entry can only be tied to one Employee entry) to refer to the User superclass, and contains the additional information.

```
class Employee(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, null=True, blank=True)
    address = models.CharField(max_length=255)
    phoneNumber = models.CharField(max_length=10)
```

As you might notice, we do not keep track of employee's role in this relation. That's because we instead keep track of that in the User relation through which Group they belong too, which also dictates what permissions they have. What it looks like for the superuser admin to edit a user's group and permissions:



Database Triggers/Signals

Django also has an equivalent system for SQL triggers called “signals”. Here are the signals that are set up on the tables to make sure there are no data de-syncs.

```
# when an inventoryVehicle is created, set the corresponding to be in inventory
@receiver(post_save, sender=InventoryVehicle)
def sync_vehicle_status_on_save(sender, instance, **kwargs):
    instance.vehicle.status = Vehicle.VehicleStatus.INVENTORY
    instance.vehicle.save()

# likewise for when an inventory vehicle is created
@receiver(post_delete, sender=InventoryVehicle)
def sync_vehicle_status_on_delete(sender, instance, **kwargs):
    instance.vehicle.status = Vehicle.VehicleStatus.IN_SERVICE
    instance.vehicle.save()

# when a sales record is created, set the corresponding vehicle to sold
@receiver(post_save, sender=Sales)
def sync_vehicle_status_on_sale(sender, instance, **kwargs):
    instance.vehicle.status = Vehicle.VehicleStatus.SOLD
    instance.vehicle.save()
```

Highlighted Features

the key features or strengths of the project that deserve special attention.
The ease-of-use and seamlessness of the django framework is a large factor of our project's implementation. The DBMS integration, web server backend, website frontend, and authentication/authorization all largely being taken care of let us focus on making sure the database schema properly encapsulated the relevant scenario of the assignment.

Other relevant features mentioned before:

- The signal-based automatic vehicle status syncing (when an InventoryVehicle is created, the parent Vehicle status updates automatically)
- The role-based permission system with the three employee groups (Mechanic, Salesperson, Billing) and what each can and can't do
- The WorkedOn relationship allowing multiple employees to be logged on a single service job with individual hours tracked
- The ability to also access the raw sqlite database, as well as a web interface with djangoql, and django's authorization implementation with "groups" creates a three-tiered database model.

Discussion and Comparison

Django was chosen as the primary framework for this project due to its "batteries included" approach. A comparable implementation in Flask, a lighter Python framework, would have required assembling these components manually from third-party libraries. Flask gives more flexibility but at the cost of significantly more setup for a full application like this one.

Outside of Python, a PHP-based implementation using Laravel could achieve a similar result, as Laravel also provides an ORM and built-in authentication (though its admin interface is less mature than Django's).

A Node.js implementation using Express would be even more barebones. Express provides almost no structure out of the box, meaning the ORM, authentication, and admin tooling would all need to be sourced and configured separately, making it not ideal for this database-centric project where the focus is schema design rather than server architecture.

On the frontend side, this project uses Django's built-in admin UI rather than a dedicated frontend framework, which was the appropriate choice given the scope. A production application would probably replace the admin UI with a custom React frontend, which would give end users a more polished experience. However, React introduces a full JavaScript build pipeline, which is quite the scope expansion for this project. Django's integrated stack kept the implementation focused on demonstrating the database development lifecycle.

Demonstration

I will run the project on localhost on a laptop when we are presenting. The SQL prompts, as well as instructions to run the project, can be found in the readme file.